Web Development Internship Report

Task 02 – Interactive Business Website

Internship Domain: Web Development

Technologies Used: HTML5, CSS3, JavaScript

Hunzla Waien

Table of Contents

Introduction 2

Objective 2

Problem Statement 2

Technologies Used 2

- HTML 2
- CSS3 3
- JavaScript 3

Implementation Steps 3

Code Screenshots and Explanations 4

- Theme Toggle 4
- Image Sliders 5
- FAQ Accordion 6
- Contact Form 7
- Animated Counters 8
- Animated Text 9

Output/Result Screenshots with explanations 10

- Theme Toggle: 10
- Image Slider 11
- FAQ Accordion 11
- Contact Form Validation 12
- Animated Counters and Typing Animation 12
- Challenges Faced 13

Repository and live demo 13

Conclusion 13

Introduction

The website was developed using HTML5, CSS3, and JavaScript. HTML was used to organize the page content, CSS was used to create the layout and visual appearance, and JavaScript was used to implement interactive features.

The project includes five main pages: Home, About, Services, Portfolio, and Contact. Together, these pages provide visitors with information about the studio, examples of its work, an overview of its services, and a way to submit project enquiries.

Objective

The objective of this task was to strengthen my front-end development skills by designing and building a fully responsive, multi-page business website using only HTML5, CSS3, and vanilla JavaScript — without relying on any frameworks or libraries. The website was required to contain five pages (Home, About, Services, Portfolio, Contact) and implement at least six interactive JavaScript features, while maintaining a consistent color scheme, modern UI/UX, and an organized folder structure.

Problem Statement

Small interior design studios need a web presence that communicates their style instantly and gives potential clients a clear way to make an enquiry. The challenge was to design a fictional studio — Wyne Interiors — from scratch: choosing a brand identity, planning five connected pages, and making the site feel alive through genuine interactivity rather than static content.

The technical problem was achieving all of this with vanilla JavaScript only. Features that are often handled by plugins (sliders, accordions, dark mode, and validation) had to be written by hand, work across all five pages from a single shared script, and remain accessible and responsive on screens from large desktops down to small phones.

Technologies Used

HTML

HTML5 was used to create the structure of the website. Each page contains the content and interface elements needed for its particular purpose, including headings, paragraphs, navigation links, images, forms, and sections.

Separate HTML files were used to keep the Home, About, Services, Portfolio, and Contact pages organized.

CSS3

CSS3 was used to control the website's appearance and layout. It defines the typography, colors, spacing, image presentation, navigation styling, buttons, cards, and other visual elements.

Responsive CSS rules were used to adapt the layout to different viewport sizes. Shared styles also help maintain a consistent appearance across the website.

JavaScript

JavaScript was used to add behavior to the static HTML pages. The script handles interface interactions, including the image slider, light and dark themes, animated counters, FAQ controls where applicable, form validation, and navigation-related behavior.

The script checks for relevant elements before attaching event listeners, allowing the same JavaScript file to support multiple pages.

Implementation Steps

Before starting the development of Wyne Interiors, the project was planned as a multi-page website for an interior design business. The main purpose was to present the business, showcase its services and completed projects, and provide a way for potential clients to make inquiries.

The planned website included the following pages and features:

- **Home:** Introduce the business and present its main highlights.
- **About:** Provide information about the business and its design approach.
- **Services:** Display the services offered and answer common questions.
- **Portfolio:** Showcase interior design projects with filtering functionality.
- **Contact:** Allow visitors to submit inquiries through a contact form.
- **User interface enhancements:** Include theme switching, animations, counters, sliders, responsive layouts, and convenient navigation features.

Code Screenshots and Explanations

Theme Toggle

This JavaScript functionality allows users to switch between the website's available visual themes. It updates the page's theme in response to user interaction.

```
// Dark Mode

const themeButton = document.querySelector(".theme-toggle");

function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;
  if (themeButton) {
    const dark = theme === "dark";

    themeButton.innerHTML = dark
      ? '<i class="fa-regular fa-sun"></i>'
      : '<i class="fa-regular fa-moon"></i>';

    themeButton.setAttribute(
      "aria-label",
      dark ? "Switch to light mode" : "Switch to dark mode",
    );
  }
}

const savedTheme = localStorage.getItem("atelier-theme") || "light";
applyTheme(savedTheme === "dark" ? "dark" : "light");

themeButton?.addEventListener("click", () => {
  const nextTheme =
    document.documentElement.dataset.theme === "dark" ? "light" : "dark";
  applyTheme(nextTheme);
  localStorage.setItem("atelier-theme", nextTheme);
});
```

Image Sliders

The slider JavaScript manages the displayed slide and responds to the available navigation controls. It allows multiple images or pieces of content to be presented within the same section.

```
//   Image Sliders

const slider = document.querySelector(".project-slider");

if (slider) {
  const slides = [...slider.querySelectorAll(".slide")];
  const dots = [...slider.querySelectorAll(".slider-dots button")];
  const previous = slider.querySelector(".slider-prev");
  const next = slider.querySelector(".slider-next");

  if (slides.length && slides.length === dots.length) {
    let current = 0;
    let autoplay;

    function showSlide(index) {
      current = (index + slides.length) % slides.length;

      slides.forEach((slide, i) => {
        const active = i === current;
        slide.classList.toggle("is-active", active);
        slide.setAttribute("aria-hidden", String(!active));
        dots[i].classList.toggle("is-active", active);

        if (active) {
          dots[i].setAttribute("aria-current", "true");
        } else {
          dots[i].removeAttribute("aria-current");
        }
      });
    }

    function stopAutoplay() {
      window.clearInterval(autoplay);
    }

    function startAutoplay() {
      stopAutoplay();
      if (!reducedMotion.matches && !document.hidden) {
        autoplay = window.setInterval(() => showSlide(current + 1), 5500);
      }
    }

    previous?.addEventListener("click", () => {
      showSlide(current - 1);
      startAutoplay();
    });

    next?.addEventListener("click", () => {
      showSlide(current + 1);
      startAutoplay();
    });

    dots.forEach((dot, index) => {
      dot.addEventListener("click", () => {
        showSlide(index);
        startAutoplay();
      });
    });

    slider.addEventListener("keydown", (event) => {
      if (event.key === "ArrowLeft" || event.key === "ArrowRight") {
        event.preventDefault();
        showSlide(current + (event.key === "ArrowRight" ? 1 : -1));
        startAutoplay();
      }
    });

    slider.addEventListener("mouseenter", stopAutoplay);
    slider.addEventListener("mouseleave", startAutoplay);
    slider.addEventListener("focusin", stopAutoplay);
    slider.addEventListener("focusout", (event) => {
      if (!slider.contains(event.relatedTarget)) startAutoplay();
    });
    document.addEventListener("visibilitychange", () => {
      document.hidden ? stopAutoplay() : startAutoplay();
    });

    startAutoplay();
  }
}
```

FAQ Accordion

The accordion functionality responds to user interaction by showing or hiding the corresponding FAQ answer. It organizes information into compact, expandable sections.

```
// FAQ

const faqButtons = document.querySelectorAll(".faq-question");

faqButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const answer = document.getElementById(
      button.getAttribute("aria-controls"),
    );
    if (!answer) return;

    const opening = button.getAttribute("aria-expanded") !== "true";

    faqButtons.forEach((otherButton) => {
      const otherAnswer = document.getElementById(
        otherButton.getAttribute("aria-controls"),
      );

      otherButton.setAttribute("aria-expanded", "false");
      otherButton.querySelector("span").textContent = "+";
      if (otherAnswer) otherAnswer.hidden = true;
    });

    button.setAttribute("aria-expanded", String(opening));
    button.querySelector("span").textContent = opening ? "−" : "+";
    answer.hidden = !opening;
  });
});
```

Contact Form

JavaScript validates the information entered into the contact form before proceeding. It helps identify missing or incorrectly formatted input and guides users toward correcting it.

```
// Form Validation

const form = document.querySelector("#contact-form");

if (form) {
  const fields = {
    name: form.querySelector("#name"),
    email: form.querySelector("#email"),
    projectType: form.querySelector("#project-type"),
    message: form.querySelector("#message"),
  };
  const status = form.querySelector("#form-status");
  const submitButton = form.querySelector('button[type="submit"]');

  const validators = {
    name(value) {
      if (!value) return "Please enter your name.";
      if (value.length < 2) return "Name must be at least 2 characters.";
      if (value.length > 60) return "Name is too long.";
      if (!/^[a-zA-ZÀ-ÿ'] -]+$/.test(value))
        return "Letters, spaces, hyphens and apostrophes only.";
      return "";
    },
    email(value) {
      if (!value) return "Please enter your email address.";
      if (!/^[^\s@]+@[^\s@]+\.[a-zA-Z]{2,}$/.test(value))
        return "Enter a valid email, e.g. name@example.com.";
      return "";
    },
    projectType(value) {
      if (!value) return "Please choose a project type.";
      return "";
    },
    message(value) {
      if (!value) return "Please tell us a little about your project.";
      if (value.length < 20)
        return `At least 20 characters — ${20 - value.length} more to go.`;
      if (value.length > 1000) return "Please keep it under 1000 characters.";
      return "";
    },
  };

  function setError(field, message) {
    const errorEl = document.getElementById(`${field.id}-error`);
    const wrapper = field.closest(".form-field");

    field.setAttribute("aria-invalid", String(Boolean(message)));
    if (errorEl) errorEl.textContent = message;
    wrapper?.classList.toggle("has-error", Boolean(message));
    wrapper?.classList.toggle(
      "is-valid",
      !message && field.value.trim() !== "",
    );

    return !message;
  }

  function validateField(key) {
    const field = fields[key];
```

Animated Counters

The counter functionality is a visual effect that animates numerical values by rapidly counting up or down until it reaches a specific target number.

```
//   Counters

const counters = document.querySelectorAll(".counter");

function animateCounter(counter) {
  if (counter.dataset.animated === "true") return;
  counter.dataset.animated = "true";
  const target = Number(counter.dataset.target);
  const suffix = counter.dataset.suffix || "";
  if (!Number.isFinite(target)) return;

  if (reducedMotion.matches) {
    counter.textContent = `${target}${suffix}`;
    return;
  }
  const start = performance.now();
  const duration = 1300;
  function update(now) {
    const progress = Math.min((now - start) / duration, 1);
    const eased = 1 - Math.pow(1 - progress, 3);
    counter.textContent = `${Math.round(target * eased)}${suffix}`;
    if (progress < 1) requestAnimationFrame(update);
  }
  requestAnimationFrame(update);
}

if (counters.length) {
  if ("IntersectionObserver" in window) {
    const observer = new IntersectionObserver(
      (entries) => {
        entries.forEach((entry) => {
          if (entry.isIntersecting) {
            animateCounter(entry.target);
            observer.unobserve(entry.target);
          }
        });
      },
      { threshold: 0.4 },
    );

    counters.forEach((counter) => observer.observe(counter));
  } else {
    counters.forEach(animateCounter);
  }
}
```

Animated Text

The typing animation is a textual effect that reveals written content progressively, mimicking the look of someone typing on a keyboard in real time.

```
// Typing Animation

const typingText = document.querySelector("#typing-text");

if (typingText && !reducedMotion.matches) {
  const phrases = (typingText.dataset.words || "").split("|").filter(Boolean);

  if (phrases.length > 1) {
    let phraseIndex = 0;
    let letterIndex = phrases[0].length;
    let deleting = true;

    const typeNext = () => {
      const phrase = phrases[phraseIndex];
      letterIndex += deleting ? -1 : 1;
      typingText.textContent = phrase.slice(0, letterIndex);

      let delay = deleting ? 48 : 85;

      if (letterIndex === 0) {
        deleting = false;
        phraseIndex = (phraseIndex + 1) % phrases.length;
        delay = 320;
      } else if (letterIndex === phrases[phraseIndex].length && !deleting) {
        deleting = true;
        delay = 2200;
      }

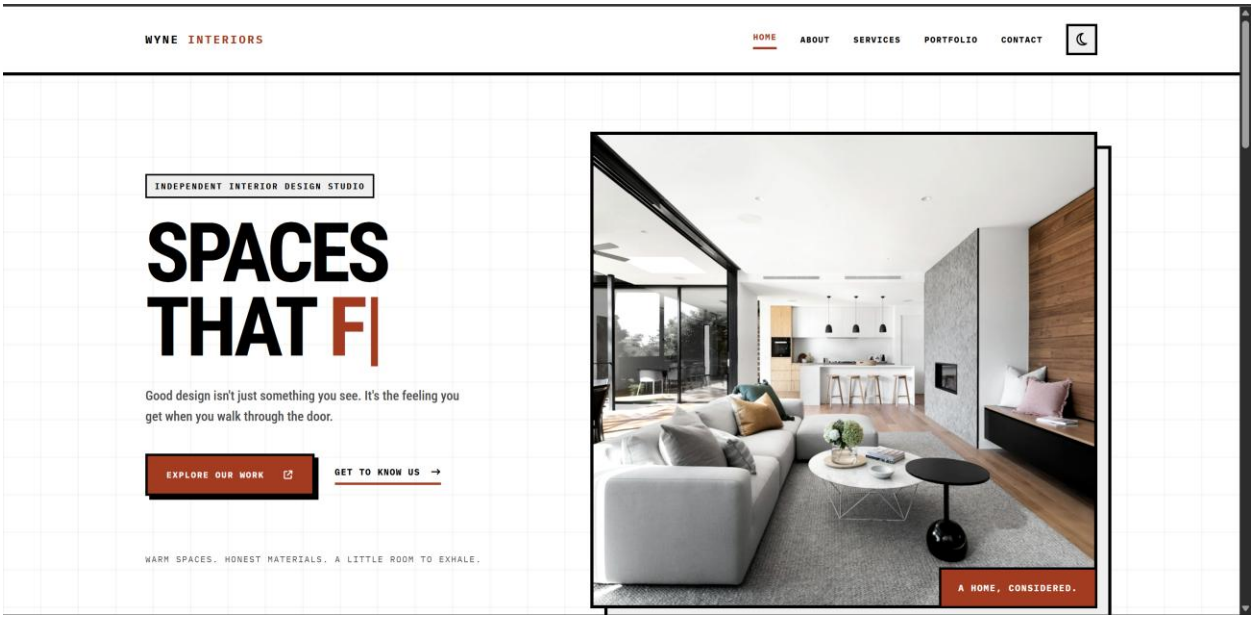
      window.setTimeout(typeNext, delay);
    };

    window.setTimeout(typeNext, 2200);
  }
}
```

Output/Result Screenshots with explanations

Theme Toggle:

The screenshots demonstrate how the website's appearance changes when the theme toggle is activated, giving visitors an alternative viewing experience.



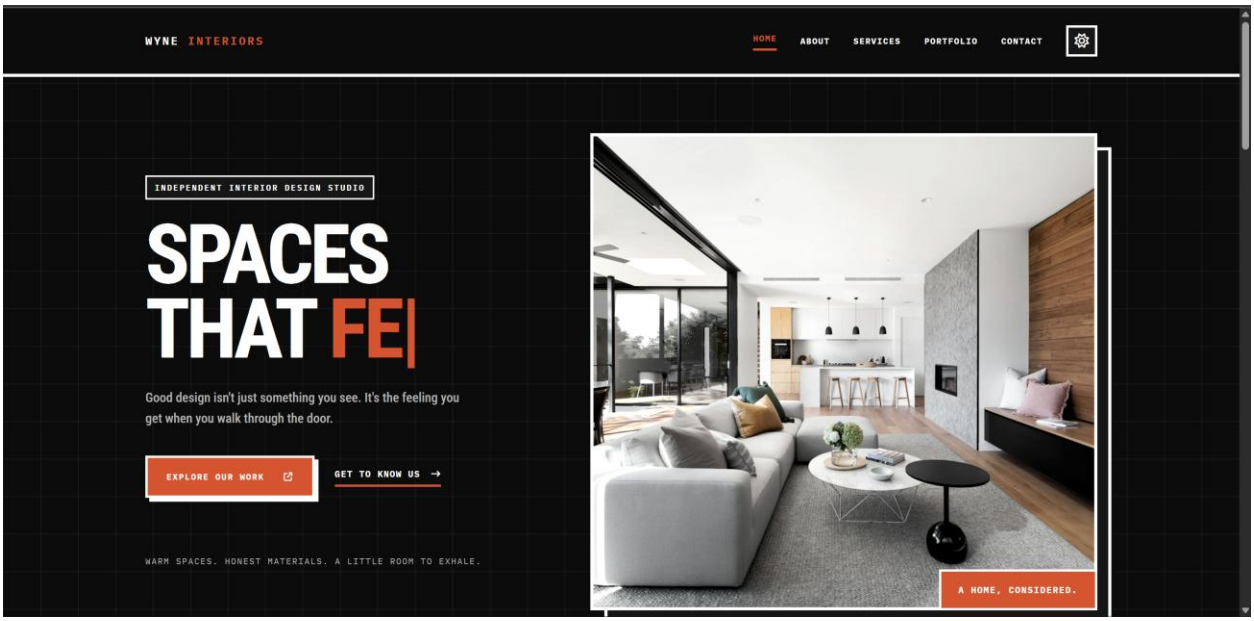
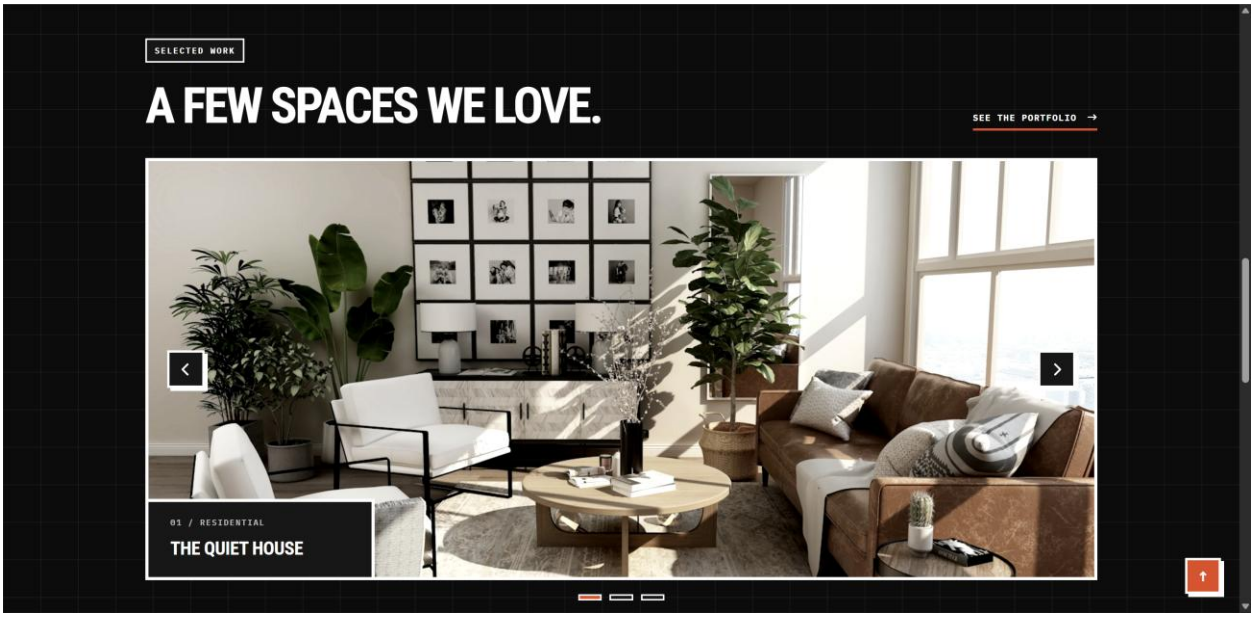


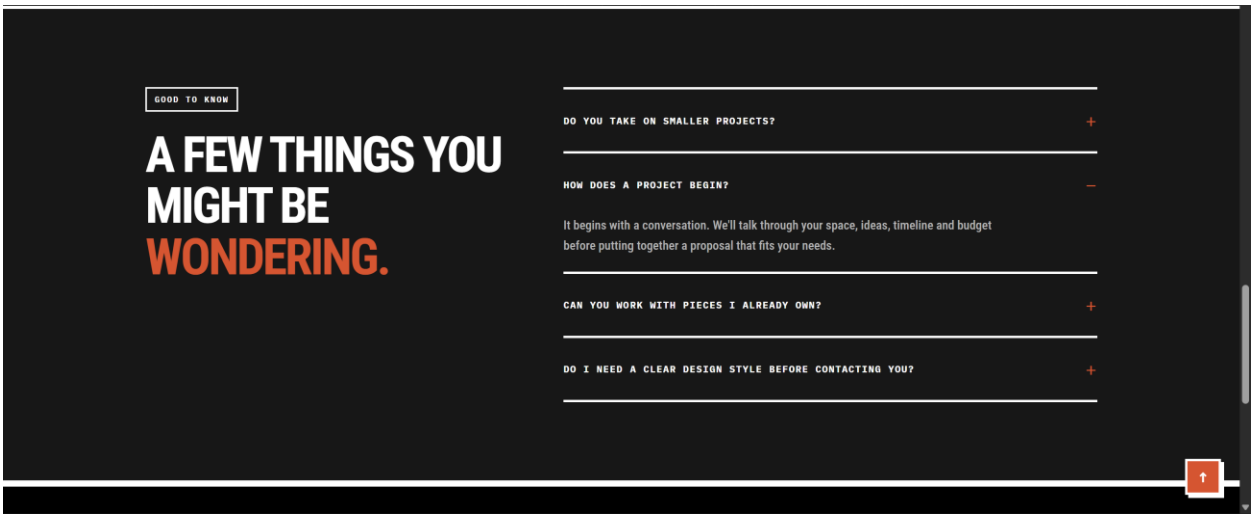
Image Slider

The output demonstrates the slider's navigation behavior and how it presents interior design visuals interactively.



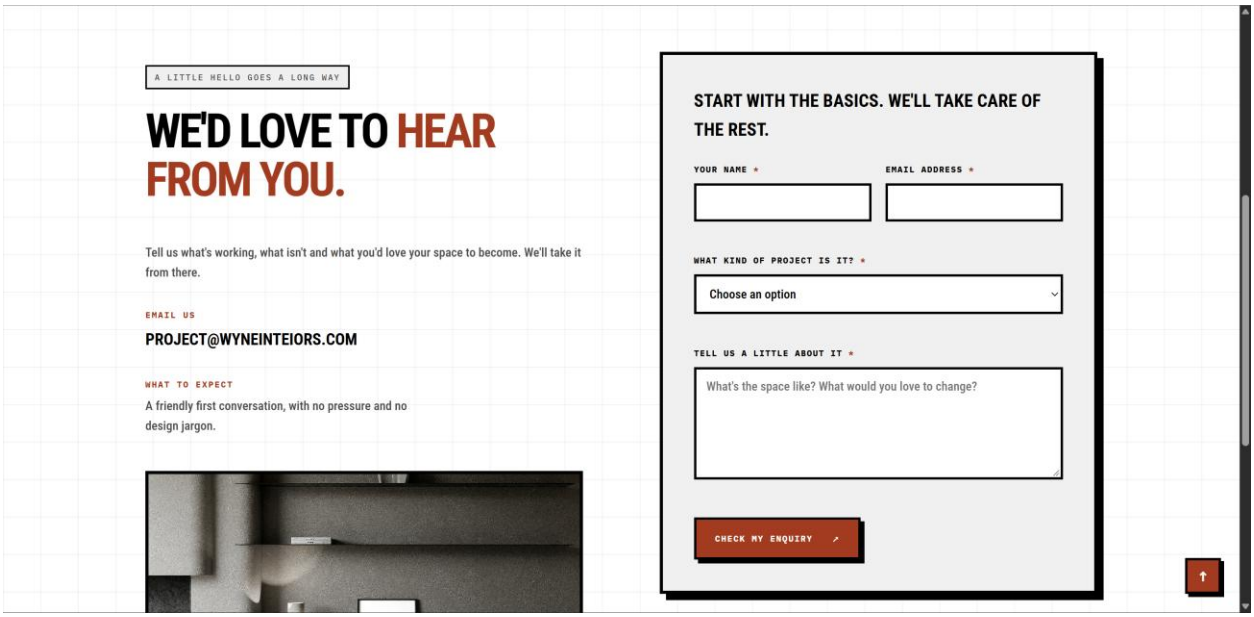
FAQ Accordion

The output demonstrates that visitors can reveal relevant answers on demand without displaying every answer at once.



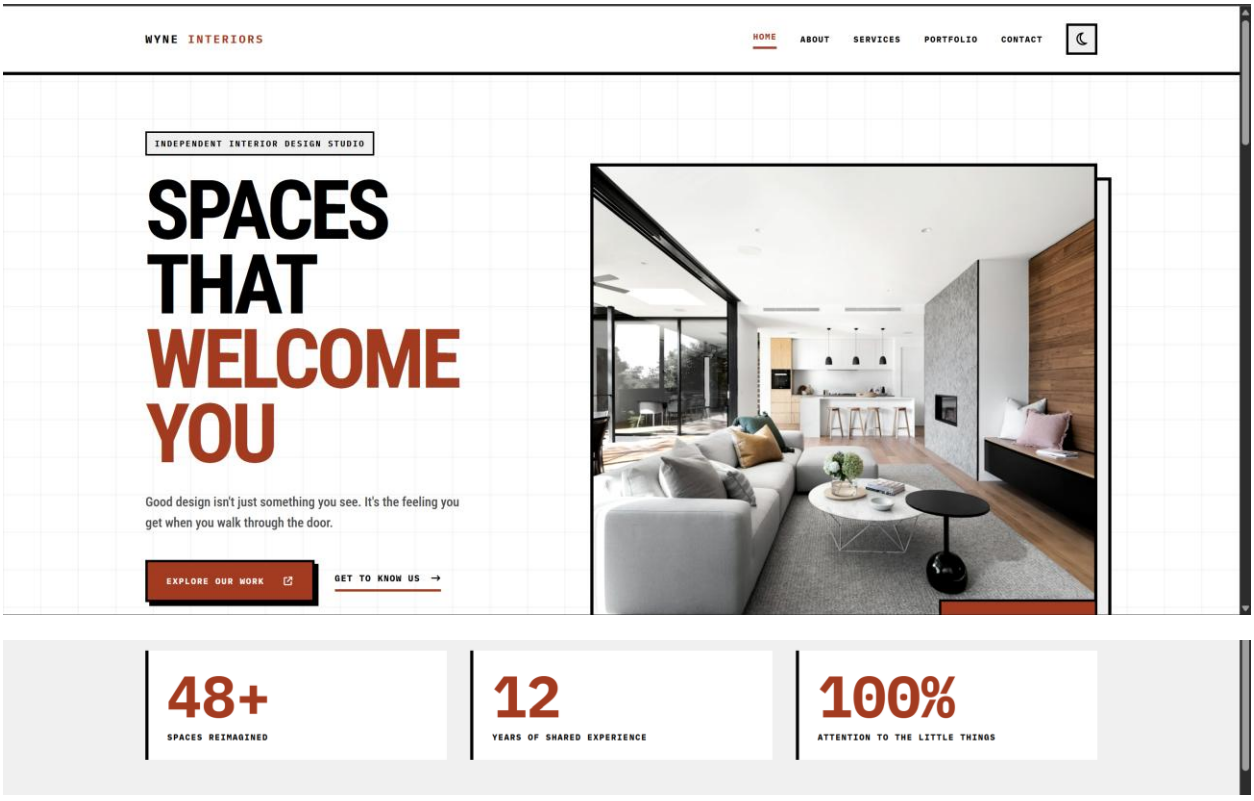
Contact Form Validation

The output demonstrates how the form responds to invalid input and communicates what needs to be corrected.



Animated Counters and Typing Animation

The output demonstrates the animated numerical highlights and text presentation used to enhance the homepage.



Challenges Faced

- Keeping the theme consistent
- Making the navigation work on mobile
- Maintaining consistency across multiple pages
- Managing interactive components
- Form Validation

Repository and live demo

Github Repository Link: https://github.com/hunzlawaien/Wyne-Interiors

GitHub Pages: https://hunzlawaien.github.io/Wyne-Interiors/index.html

Conclusion

This project demonstrates the use of HTML5, CSS3, and JavaScript to build a multi-page website for an interior design studio.

The project brings together structured page content, a shared visual design, project imagery, and interactive features. The Home, About, Services, Portfolio, and Contact pages each serve a distinct purpose while contributing to a consistent user experience.